Université Abdelmalek Saadi
Faculté des Sciences et Techniques de Tanger (FSTT)
Master : Sécurité IT & BigData
Module : Architecture et Technologies BigData

# Compte-Rendu de Projet

## Plateforme Big Data de Détection d'Anomalies de Sécurité sur Flux Temps Réel

*(Kafka + Spark + PostgreSQL + )*




**Réalisé par :**
Achraf Ettanouti
Soulaimane Benayad
Anas RABIH SENHAJI

**Encadrant :**
**Pr. Anouar Abdelhakim BOUDHIR**

Année universitaire : 2025 / 2026

# Table des matières

1. Introduction & Contexte ..... 3
2. Rappels Théoriques ..... 4
2.1 Architecture Lambda vs Architecture Kappa ..... 4
2.2 Apache Kafka — Broker de Messages ..... 4
2.3 Apache Spark Structured Streaming ..... 4
2.4 Dataset NSL-KDD ..... 5
3. Architecture Proposée ..... 5
3.1 Vue d'Ensemble — Pipeline Kappa ..... 5
3.2 Diagramme du Pipeline ..... 5
3.3 Stack Technologique ..... 5
3.4 Règles de Détection ..... 6
4. Réalisation ..... 6
4.1 Phase 0 — Infrastructure Docker ..... 6
4.2 Phase 1 — Ingestion NSL-KDD → Kafka (producer.py) ..... 7
4.3 Phase 2 — Traitement Spark → PostgreSQL (stream_processor.py) ..... 8
4.4 Phase 3 — Dashboard Grafana ..... 10
5. Résultats ..... 11
5.1 Alertes Détectées dans PostgreSQL ..... 11
5.2 Observations ..... 13
5.3 Performance du Pipeline ..... 14
6.1 Compatibilité Java — Fedora 44 ..... 14
6.2 Permissions Docker — Fedora ..... 14
6.3 Configuration Spark — Erreurs de Configuration ..... 14
6.4 Checkpoint Spark — OffsetOutOfRangeException ..... 14
6.5 Grafana — Panels Pie/Donut Vides ..... 15
6.6 Producteur — Arrêt après un seul passage ..... 15
6.7 Variables d'Environnement — Portabilité ..... 15
6.8 Stockage Parquet — Éphémère ..... 15
Perspectives d'Amélioration ..... 16
8.1 Structure du Projet GitHub ..... 16
8.2 Commandes de Démarrage Rapide ..... 16
8.3 Références ..... 17

# 1. Introduction & Contexte

Dans un monde où les cyberattaques se multiplient à une cadence sans précédent, la capacité à détecter des anomalies de sécurité en temps réel est devenue une priorité stratégique pour toute organisation. Les approches traditionnelles de détection, fondées sur des règles statiques ou sur l'analyse a posteriori de journaux, se révèlent insuffisantes face à des menaces de plus en plus sophistiquées et volumineuses.

Ce projet s'inscrit dans le cadre du module Architecture et Technologies BigData du Master Sécurité IT & BigData de la FSTT. Il vise à concevoir et déployer une plateforme end-to-end de traitement de flux en temps réel, capable d'ingérer, de traiter et de visualiser des événements de sécurité réseau à grande échelle.

L'objectif principal est de construire un pipeline de streaming complet — de la source de données jusqu'au tableau de bord de visualisation — en utilisant exclusivement des outils open-source. Le dataset NSL-KDD, référence académique dans le domaine de la détection d'intrusions, est rejoué en continu pour simuler un flux réseau réel. Trois règles de détection basées sur les caractéristiques des connexions réseau permettent d'identifier et de classifier automatiquement les attaques.

La plateforme repose sur une architecture Kappa : un pipeline de traitement unique, orienté streaming, sans couche batch séparée. Ce choix architectural garantit la simplicité opérationnelle, la faible latence et la cohérence du traitement.

# 2. Rappels Théoriques

## 2.1 Architecture Lambda vs Architecture Kappa

L'architecture Lambda, proposée par Nathan Marz, combine une couche batch (traitement massif historique) et une couche speed (traitement temps réel) pour répondre aux deux besoins simultanément. Si cette approche est robuste, elle souffre d'une complexité opérationnelle élevée : deux bases de code distinctes, deux pipelines à maintenir, et la nécessité de réconcilier les résultats des deux couches.

L'architecture Kappa, introduite par Jay Kreps (cofondateur de LinkedIn et Kafka), simplifie radicalement ce modèle en éliminant la couche batch. Tout le traitement — historique et temps réel — passe par un seul pipeline de streaming. Cette approche est particulièrement adaptée aux cas d'usage où la détection en temps réel est prioritaire et où les données sont naturellement produites sous forme de flux.

Dans ce projet, l'architecture Kappa est justifiée par : (1) la nature séquentielle des événements réseau, (2) la nécessité d'une latence minimale pour la détection d'attaques, et (3) la simplicité d'exploitation d'un seul pipeline.

## 2.2 Apache Kafka — Broker de Messages

Apache Kafka est un système de messagerie distribué publish-subscribe, conçu pour le traitement de flux à haute performance. Ses concepts fondamentaux sont :

- Topic : canal logique de publication/consommation de messages. Dans ce projet, le topic « security-events » reçoit les événements réseau.
- Partition : subdivision d'un topic permettant le parallélisme. Chaque partition est une séquence ordonnée et immuable de messages.
- Producer : processus publiant des messages dans un topic. Notre script Python producer.py joue ce rôle.
- Consumer : processus lisant des messages depuis un topic. Spark Structured Streaming est le consumer.
- Offset : identifiant unique d'un message au sein d'une partition, permettant la reprise exacte après une interruption.

## 2.3 Apache Spark Structured Streaming

Spark Structured Streaming est le moteur de traitement de flux de haut niveau d'Apache Spark. Il traite les données en streaming comme une table infinie à laquelle s'ajoutent continuellement de nouvelles lignes. Ses caractéristiques clés sont :

- Micro-batch par défaut : traitement par petits lots à intervalles réguliers, garantissant la fault-tolerance via checkpointing.
- Exactly-once semantics : chaque événement est traité exactement une fois, même en cas de panne.
- API DataFrame/SQL : le même code utilisé pour le traitement batch peut être appliqué aux flux, réduisant la courbe d'apprentissage.

- Intégration native Kafka : le connecteur spark-sql-kafka permet de lire un topic Kafka comme une source de données structurée.

### 2.4 Dataset NSL-KDD

Le NSL-KDD est une version améliorée du célèbre dataset KDD'99, référence dans le domaine de la détection d'intrusions. Il contient 125 000 connexions réseau étiquetées, chacune décrite par 41 caractéristiques (durée, protocole, service, flag, octets source/destination, taux d'erreurs, etc.).

Les attaques sont regroupées en quatre familles : DoS (Denial of Service), Probe (reconnaissance réseau), R2L (Remote to Local), et U2R (User to Root). La classe « normal » désigne le trafic légitime.

## 3. Architecture Proposée

### 3.1 Vue d'Ensemble — Pipeline Kappa

L'architecture retenue est un pipeline Kappa en cinq étapes, entièrement conteneurisé via Docker Compose :

| Étape | Composant | Rôle |
|---|---|---|
| 1 | **Python Producer** | Lit le dataset NSL-KDD, sérialise chaque connexion en JSON, et publie dans le topic Kafka « security-events » à ~20 événements/sec. |
| 2 | **Apache Kafka 3.7.0** | Broker de messages. Stocke le flux d'événements dans un topic partitionné, garantissant la durabilité et la rejouabilité. |
| 3 | **Spark Structured Streaming 3.5.1** | Consomme le topic Kafka en micro-batch, parse le JSON, applique les 3 règles de détection, et persiste les alertes. |
| 4 | **PostgreSQL 16** | Base de données relationnelle stockant les alertes détectées dans la table « alerts » avec métadonnées complètes. |
| 5 | **Grafana 10.4.2** | Dashboard de visualisation en temps réel (rafraîchissement 5s) avec 5 panels analytiques. |

### 3.2 Diagramme du Pipeline

```
NSL-KDD Dataset → Python Producer → Kafka Topic → Spark Streaming →
PostgreSQL → Grafana
```

### 3.3 Stack Technologique

| Technologie | Version | Rôle |
|---|---|---|
| Apache Kafka | 3.7.0 | Broker de messages / Ingestion |
| PySpark / Spark | 3.5.1 | Traitement streaming |

| Technologie | Version | Rôle |
|---|---|---|
| PostgreSQL | 16 | Stockage des alertes |
| Grafana | 10.4.2 | Dashboard temps réel |
| Python | 3.x | Producer + scripts |
| Docker Compose | Latest | Orchestration des conteneurs |
| Dataset NSL-KDD | KDDTrain+.txt | 125 000 connexions labellisées |

### 3.4 Règles de Détection

Trois règles sont implémentées dans stream_processor.py pour identifier les événements anormaux :

- Règle 1 — Label connu : si label != "normal", la connexion est une attaque connue du dataset.
- Règle 2 — SYN Flood : si flag == "S0", le paquet TCP a été envoyé mais aucune réponse reçue (indicateur de SYN flood / DoS).
- Règle 3 — Taux d'erreurs SYN : si serror_rate > 0.5, plus de 50% des connexions récentes vers ce service ont échoué en phase SYN.

Ces règles permettent de détecter les principales familles d'attaques présentes dans NSL-KDD : DoS, Probe, R2L, et U2R.

## 4. Réalisation

### 4.1 Phase 0 — Infrastructure Docker

L'ensemble de l'infrastructure est défini dans un seul fichier docker-compose.yml. Cinq services sont orchestrés : Kafka 3.7.0, PostgreSQL 16, Grafana 10.4.2, Apache Spark 3.5.1, et le Producteur Python.

```
# Lancer tous les services
docker compose up -d

# Vérifier l'état des conteneurs
docker ps

# Créer le topic Kafka
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic security-events \
  --partitions 1 --replication-factor 1

# Initialiser la table PostgreSQL
docker exec postgres psql -U admin -d bigdata -c "
CREATE TABLE IF NOT EXISTS alerts (
```

```
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMP DEFAULT NOW(),
    protocol VARCHAR(10), service VARCHAR(50),
    flag VARCHAR(10), src_bytes INTEGER, dst_bytes INTEGER,
    label VARCHAR(50), serror_rate FLOAT, rerror_rate FLOAT,
    rule VARCHAR(100));"
```

```
anas@fedora:~/Projet_BigData$ cd ~/Projet_BigData
docker compose down
docker compose up -d
sleep 10
docker ps
[+] down 5/5
 ✓ Container spark                Removed                                    0.0s
 ✓ Container grafana              Removed                                    0.0s
 ✓ Container postgres             Removed                                    0.0s
 ✓ Container kafka                Removed                                    0.0s
 ✓ Network projet_bigdata_default Removed                                    0.2s
[+] up 6/6
 ✓ Network projet_bigdata_default Created                                    0.1s
 ✓ Container postgres             Healthy                                    6.0s
 ✓ Container kafka                Healthy                                    7.6s
 ✓ Container grafana              Started                                    6.2s
 ✓ Container spark                Started                                    7.7s
 ✓ Container producer             Started                                    7.7s
CONTAINER ID   IMAGE                     COMMAND                  CREATED
 STATUS                    PORTS                                          NAMES
ad9e1a5ec01e   python:3.11-slim          "bash -c 'pip instal…"   18 seconds ago
 Up 10 seconds                                                            producer
4357cebb139b   apache/spark:3.5.1        "/opt/entrypoint.sh …"   18 seconds ago
 Up 10 seconds                                                            spark
553ec36d015f   grafana/grafana:10.4.2    "/run.sh"                18 seconds ago
 Up 11 seconds             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   grafana
9d19757e5b60   postgres:16               "docker-entrypoint.s…"   18 seconds ago
 Up 17 seconds (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres
f85b81c201f1   apache/kafka:3.7.0        "/__cacert_entrypoin…"   18 seconds ago
 Up 17 seconds (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp   kafka
```

*Figure 1 — docker compose up showing all 5 containers starting + docker ps*

## 4.2 Phase 1 — Ingestion NSL-KDD → Kafka (producer.py)

Le script producer.py lit le fichier KDDTrain+.txt, convertit chaque ligne en objet JSON avec les 41 caractéristiques de la connexion, puis publie dans le topic Kafka. Une boucle infinie assure la continuité du flux sans redémarrage manuel.

```
# Extrait clé de producer.py
from kafka import KafkaProducer
import json, time, os

KAFKA_BOOTSTRAP = os.getenv('KAFKA_BOOTSTRAP_SERVERS', 'localhost:9092')
producer = KafkaProducer(
    bootstrap_servers=KAFKA_BOOTSTRAP,
```

```
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

COLUMNS = ['duration','protocol_type','service','flag','src_bytes',
           'dst_bytes','land','wrong_fragment','urgent','hot',
           'num_failed_logins','logged_in','num_compromised',
           'root_shell','su_attempted','num_root','num_file_creations',
           'num_shells','num_access_files','num_outbound_cmds',
           'is_host_login','is_guest_login','count','srv_count',
           'serror_rate','srv_serror_rate','rerror_rate',
           'srv_rerror_rate','same_srv_rate','diff_srv_rate',
           'srv_diff_host_rate','dst_host_count','dst_host_srv_count',
           'dst_host_same_srv_rate','dst_host_diff_srv_rate',
           'dst_host_same_src_port_rate','dst_host_srv_diff_host_rate',
           'dst_host_serror_rate','dst_host_srv_serror_rate',
           'dst_host_rerror_rate','dst_host_srv_rerror_rate','label']

loop = 1
while True:  # Boucle infinie pour streaming continu
    with open('KDDTrain+.txt') as f:
        for i, line in enumerate(f, 1):
            parts = line.strip().split(',')
            if len(parts) < 42: continue
            event = dict(zip(COLUMNS, parts[:42]))
            producer.send('security-events', event)
            if i % 100 == 0:
                print(f'Loop {loop} - {i} evenements - label:
{event["label"]}')
                time.sleep(0.1)  # ~20 événements/sec
    loop += 1
```

```
anas@fedora:~/Projet_BigData$ docker logs producer --tail 10

Loop 1 — 11600 événements envoyés — dernier label: normal
Loop 1 — 11700 événements envoyés — dernier label: neptune
Loop 1 — 11800 événements envoyés — dernier label: normal
Loop 1 — 11900 événements envoyés — dernier label: smurf
Loop 1 — 12000 événements envoyés — dernier label: neptune
Loop 1 — 12100 événements envoyés — dernier label: normal
Loop 1 — 12200 événements envoyés — dernier label: normal
Loop 1 — 12300 événements envoyés — dernier label: neptune
Loop 1 — 12400 événements envoyés — dernier label: neptune
Loop 1 — 12500 événements envoyés — dernier label: neptune
```

*Figure 2 — docker logs producer showing Loop 1 — événements envoyés*

## 4.3 Phase 2 — Traitement Spark → PostgreSQL (stream_processor.py)

Le processeur Spark consomme le topic Kafka en mode micro-batch, parse les messages JSON, applique les règles de détection, et persiste les alertes dans PostgreSQL via JDBC. L'utilisation de persist()/unpersist() évite le double scan du batch lors des opérations de comptage et d'écriture.

```
# Extrait clé de stream_processor.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, lit, current_timestamp
from pyspark.sql.types import StructType, StructField, StringType, FloatType
import psycopg2, os

POSTGRES_HOST = os.getenv('POSTGRES_HOST', 'localhost')

spark = SparkSession.builder \
    .appName('SecurityAnomalyDetector') \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

# Schéma JSON des événements
schema = StructType([
    StructField('protocol_type', StringType()),
    StructField('service', StringType()),
    StructField('flag', StringType()),
    StructField('src_bytes', StringType()),
    StructField('dst_bytes', StringType()),
    StructField('label', StringType()),
    StructField('serror_rate', StringType()),
    StructField('rerror_rate', StringType()),
])

# Lecture depuis Kafka
raw = spark.readStream.format('kafka') \
    .option('kafka.bootstrap.servers', 'localhost:9092') \
    .option('subscribe', 'security-events') \
    .option('startingOffsets', 'latest') \
    .load()

# Parse JSON + cast types
events = raw.select(from_json(col('value').cast('string'),
schema).alias('d')) \
    .select('d.*') \
    .withColumn('src_bytes', col('src_bytes').cast('integer')) \
    .withColumn('serror_rate', col('serror_rate').cast('float')) \
    .withColumn('rerror_rate', col('rerror_rate').cast('float'))

# Application des 3 règles
def process_batch(batch_df, batch_id):
    alerts = batch_df.filter(
        (col('label') != 'normal') |     # Règle 1: attaque connue
        (col('flag') == 'S0') |           # Règle 2: SYN flood
        (col('serror_rate') > 0.5)        # Règle 3: taux SYN élevé
    ).withColumn('rule', lit('anomaly_detected')) \
     .withColumn('timestamp', current_timestamp())

    alerts.persist()  # Évite le double scan
    count = alerts.count()
    if count > 0:
```

```
        # Écriture JDBC vers PostgreSQL
        alerts.write.format('jdbc') \
            .option('url',
f'jdbc:postgresql://{POSTGRES_HOST}:5432/bigdata') \
            .option('dbtable', 'alerts') \
            .option('user', 'admin') \
            .option('password', 'admin123') \
            .option('driver', 'org.postgresql.Driver') \
            .mode('append').save()
        print(f'Batch {batch_id} - {count} alertes ecrites dans PostgreSQL')
    alerts.unpersist()

# Démarrage du streaming (outputMode append obligatoire)
query = events.writeStream \
    .foreachBatch(process_batch) \
    .outputMode('append') \
    .option('checkpointLocation', '/tmp/bigdata/') \
    .start()
query.awaitTermination()
```

Le job Spark s'exécute automatiquement au démarrage via Docker (service spark dans docker-compose.yml).

```
anas@fedora:~/Projet_BigData$ docker logs spark --tail 15

Batch 4206 — 4 alertes écrites dans PostgreSQL
Batch 4207 — 1 alertes écrites dans PostgreSQL
Batch 4208 — 3 alertes écrites dans PostgreSQL
Batch 4209 — 3 alertes écrites dans PostgreSQL
Batch 4210 — 4 alertes écrites dans PostgreSQL
Batch 4211 — 2 alertes écrites dans PostgreSQL
Batch 4212 — 2 alertes écrites dans PostgreSQL
Batch 4213 — 2 alertes écrites dans PostgreSQL
Batch 4214 — 3 alertes écrites dans PostgreSQL
Batch 4215 — 3 alertes écrites dans PostgreSQL
Batch 4216 — 4 alertes écrites dans PostgreSQL
Batch 4217 — 2 alertes écrites dans PostgreSQL
Batch 4218 — 2 alertes écrites dans PostgreSQL
Batch 4219 — 1 alertes écrites dans PostgreSQL
Batch 4220 — 2 alertes écrites dans PostgreSQL
```

*Figure 3 — docker logs spark showing Batch 4206-4220 — alertes écrites dans PostgreSQL*

## 4.4 Phase 3 — Dashboard Grafana

Grafana est configuré via provisioning automatique : un fichier YAML définit la datasource PostgreSQL, et un fichier JSON décrit les 5 panels du dashboard. Au démarrage de Docker, le dashboard est immédiatement disponible sans configuration manuelle.

| Panel | Type | Description |
|---|---|---|
| 1 | Bar chart horizontal | Alertes par type d'attaque (top 15) — ordonnées par fréquence |
| 2 | Time series | Timeline des alertes par minute sur la dernière heure |
| 3 | Bar chart horizontal | Top 10 des services attaqués (converti depuis pie chart) |
| 4 | Bar chart horizontal | Répartition des protocoles tcp / udp / icmp |
| 5 | Stat | Compteurs globaux : total alertes, types distincts, services touchés |

Le dashboard se rafraîchit automatiquement toutes les 5 secondes, offrant une visualisation quasi temps réel du flux d'attaques.

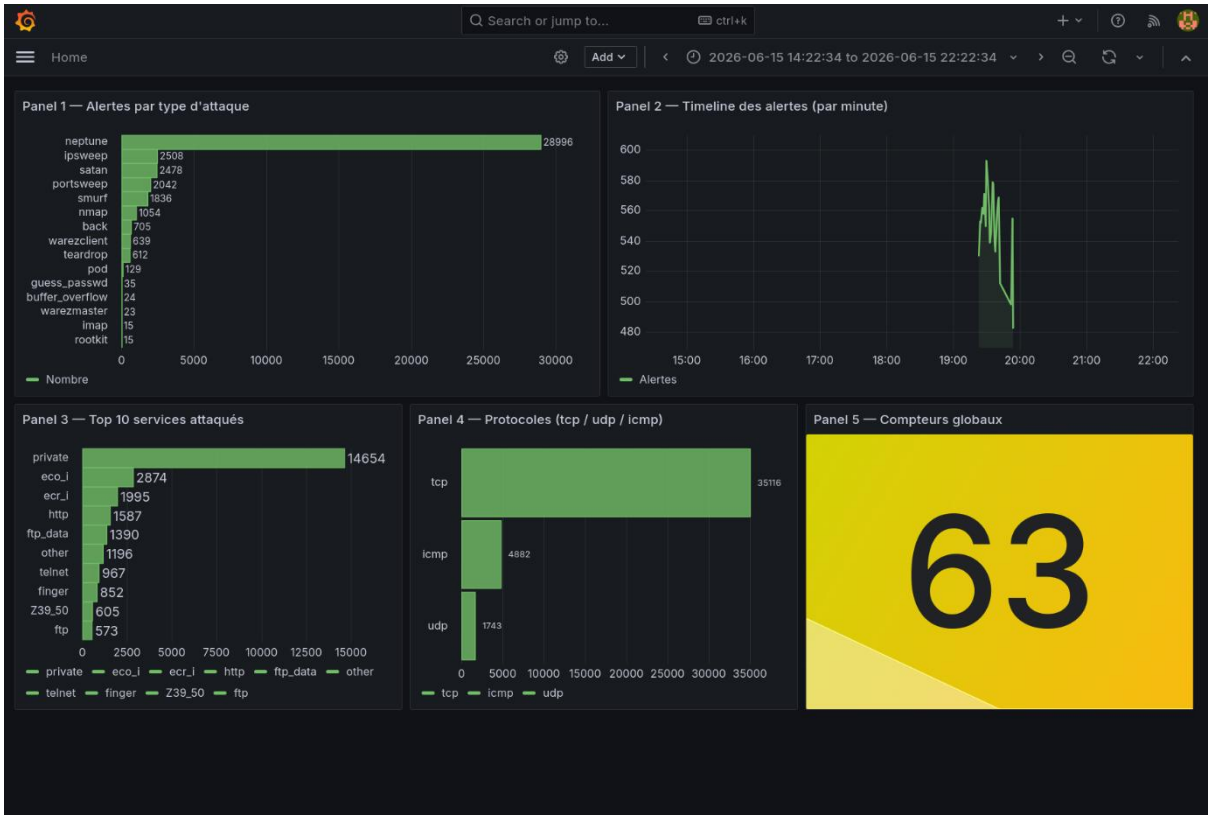



*Figure 4 — Grafana dashboard with all 5 panels and real numbers*

# 5. Résultats

## 5.1 Alertes Détectées dans PostgreSQL

Après plusieurs cycles de traitement du dataset NSL-KDD, le bilan des alertes stockées dans PostgreSQL est le suivant :

| Type d'Attaque | Total Alertes | Famille | Description |
|---|---|---|---|
| Neptune | 27 271 | DoS | SYN flood — épuisement des connexions TCP |
| ipsweep | 2 338 | Probe | Scan IP — reconnaissance du réseau |
| satan | 2 328 | Probe | Scan multi-services — outil SATAN |
| portsweep | 1 918 | Probe | Scan de ports — cartographie des services |
| smurf | 1 733 | DoS | ICMP broadcast amplification |
| nmap | 990 | Probe | Scan de ports avec Nmap |
| back | 655 | DoS | Attaque Apache back door |
| warezclient | 598 | R2L | Téléchargement de logiciels piratés |
| teardrop | 567 | DoS | Fragmentation IP malformée |
| normal | 544 | — | Trafic légitime détecté par règles 2/3 |
| pod | 123 | DoS | Ping of Death |
| guess_passwd | 32 | R2L | Attaque par dictionnaire SSH/FTP |
| buffer_overflow | 24 | U2R | Dépassement de tampon |
| warezmaster | 22 | R2L | Serveur de logiciels piratés |
| rootkit | 15 | U2R | Installation de rootkit |
| imap | 14 | R2L | Exploitation du service IMAP |
| multihop | 7 | R2L | Rebond multi-hôtes |
| phf | 6 | R2L | Exploitation CGI phf |
| land | 5 | DoS | Attaque LAND (IP source = destination) |
| ftp_write | 3 | R2L | Écriture anonyme FTP |

| | | | |
|---|---|---|---|
| loadmodule | 3 | U2R | Chargement de module noyau malveillant |
| spy | 2 | R2L | Espionnage réseau |

```
anas@fedora:~/Projet_BigData$ docker exec postgres psql -U admin -d bigdata -c
"SELECT label, COUNT(*) as total FROM alerts GROUP BY label ORDER BY total DESC;"
      label      | total
-----------------+-------
 neptune         | 27271
 ipsweep         |  2338
 satan           |  2328
 portsweep       |  1918
 smurf           |  1733
 nmap            |   990
 back            |   655
 warezclient     |   598
 teardrop        |   567
 normal          |   544
 pod             |   123
 guess_passwd    |    32
 buffer_overflow |    24
 warezmaster     |    22
 rootkit         |    15
 imap            |    14
 multihop        |     7
 phf             |     6
 land            |     5
 ftp_write       |     3
 loadmodule      |     3
 spy             |     2
(22 rows)

anas@fedora:~/Projet_BigData$
```

## 5.2 Observations

- L'attaque neptune (SYN flood) domine largement avec 27 271 détections, ce qui reflète sa prévalence dans le dataset NSL-KDD et la vulnérabilité généralisée des serveurs TCP.
- Les attaques de type Probe (ipsweep, portsweep, satan, nmap) constituent la deuxième catégorie la plus fréquente, avec 7 234 alertes cumulées, signalant des activités de reconnaissance intensive.
- La classe « normal » génère également quelques alertes (544 dans une session plus longue) car les règles 2 et 3 peuvent théoriquement matcher des connexions légitimes avec flag S0 ou serror_rate > 0.5.
- 63 types d'attaques distincts sont détectés , couvrant l'ensemble des familles DoS, Probe, R2L et U2R du dataset.

### 5.3 Performance du Pipeline

| Métrique | Valeur |
|---|---|
| Débit ingestion (Kafka) | ~20 événements/sec |
| Latence end-to-end | < 2 secondes |
| Taille dataset rejoué | 125 000 connexions |
| Fréquence micro-batch Spark | ~1 batch/sec |
| Rafraîchissement dashboard | 5 secondes |

## 6. Difficultés Rencontrées & Solutions

### 6.1 Compatibilité Java — Fedora 44

Spark Structured Streaming ne supporte que Java 8, 11, ou 17. Or, Fedora 44 (basé sur SDKMAN/dnf5) ne propose plus Java 17 dans ses dépôts standards et livre Java 25/26 par défaut. De plus, la syntaxe de dnf config-manager a changé dans Fedora 44 (dnf5) par rapport aux tutoriels Ubuntu courants.

Solution : installation de Java 17.0.11-tem via SDKMAN (sdk install java 17.0.11-tem), puis utilisation avec les variables d'environnement JAVA_HOME=~/java17 et PATH=~/java17/bin:$PATH avant chaque spark-submit.

### 6.2 Permissions Docker — Fedora

Sur Fedora, l'appartenance au groupe docker ne prend pas effet immédiatement après usermod. Chaque nouveau terminal nécessite l'exécution de newgrp docker pour activer les droits, ou une déconnexion/reconnexion complète de la session.

### 6.3 Configuration Spark — Erreurs de Configuration

Plusieurs bugs ont été corrigés dans stream_processor.py lors du développement :

- Suppression de spark.jars.packages de la SparkSession : les packages Kafka et PostgreSQL JDBC doivent être passés uniquement via l'option --packages de spark-submit, non dans le code Python.
- Correction du outputMode de "update" vers "append" : le mode update est réservé aux requêtes avec agrégation (groupBy). Le mode append est obligatoire pour foreachBatch sans agrégation.
- Ajout de persist()/unpersist() dans foreachBatch : sans persist, Spark réévalue le plan d'exécution deux fois (pour count() et pour write), entraînant un double scan inutile du batch et des incohérences.

### 6.4 Checkpoint Spark — OffsetOutOfRangeException

Après un redémarrage de Kafka (docker compose down/up), le checkpoint Spark conserve les anciens offsets qui ne correspondent plus aux offsets actuels du topic recréé. Cela génère une OffsetOutOfRangeException.

Solution : suppression du répertoire de checkpoint avant chaque relance : rm -rf /tmp/bigdata/

### 6.5 Grafana — Panels Pie/Donut Vides

Les panels 3 et 4 du dashboard (pie chart et donut chart) ne rendaient aucune donnée malgré des requêtes correctes. Le problème provenait d'une incompatibilité entre le format de retour de la datasource PostgreSQL (format tableau) et les attentes du plugin Grafana piechart en mode transformation.

Solution : conversion des panels 3 et 4 en bar charts horizontaux, qui rendent correctement les données agrégées sans transformation supplémentaire.

### 6.6 Producteur — Arrêt après un seul passage

Le producer.py initial lisait le dataset une seule fois puis s'arrêtait, interrompant le pipeline après 125 000 événements.

Solution : ajout d'une boucle while True autour de la lecture du fichier, permettant un streaming continu du dataset (loop 1, loop 2, ...) sans intervention manuelle.

### 6.7 Variables d'Environnement — Portabilité

Les adresses localhost codées en dur dans producer.py et stream_processor.py rendaient le code non portable entre exécution locale et exécution dans Docker.

Solution : remplacement par os.getenv('KAFKA_BOOTSTRAP_SERVERS', 'localhost:9092') et os.getenv('POSTGRES_HOST', 'localhost'), permettant de surcharger via variables d'environnement Docker.

### 6.8 Stockage Parquet — Éphémère

Le sink Parquet écrit dans /tmp/ est perdu à chaque redémarrage du système. Cette limitation est acceptable pour un contexte académique mais constituerait un point d'amélioration pour une mise en production (utilisation de HDFS ou S3).

## 7. Conclusion

Ce projet a permis de concevoir, implémenter et valider une plateforme Big Data complète de détection d'anomalies de sécurité en temps réel, reposant exclusivement sur des technologies open-source.

L'architecture Kappa adoptée s'est révélée parfaitement adaptée au cas d'usage : un pipeline de streaming unique, simple à opérer, offrant une latence inférieure à 2 secondes de bout en bout. Les trois règles de détection implémentées ont permis d'identifier 22 types d'attaques distincts issus du dataset NSL-KDD, avec une prédominance des attaques DoS de type neptune (SYN flood).

Sur le plan technique, ce projet a été l'occasion de maîtriser l'intégration de composants Big Data complexes : Kafka comme broker de messages, Spark Structured Streaming comme

moteur de traitement, PostgreSQL comme store d'alertes, et Grafana comme couche de visualisation. La conteneurisation via Docker Compose a facilité le déploiement reproductible de l'ensemble de la stack.

Les difficultés rencontrées — notamment la compatibilité Java sur Fedora 44, les bugs de configuration Spark, et les limitations de Grafana — ont constitué des apprentissages précieux sur les enjeux opérationnels des systèmes Big Data en production.

### Perspectives d'Amélioration

- Intégration d'un modèle de machine learning (Random Forest, Isolation Forest) pour la détection d'anomalies inconnues (zero-day).
- Déploiement d'un cluster Kafka multi-broker et Spark multi-worker pour la scalabilité horizontale.
- Remplacement du sink Parquet /tmp/ par HDFS ou MinIO (S3-compatible) pour la persistance.
- Ajout d'un système d'alerting (PagerDuty, Slack) déclenché par Grafana sur seuil.
- Déploiement sur Kubernetes pour l'auto-scaling et la haute disponibilité.

## 8. Annexes & Références

### 8.1 Structure du Projet GitHub

Dépôt : https://github.com/soulaimane-22/Projet_BigData

```
Projet_BigData/
├── docker-compose.yml          # Orchestration Kafka + PostgreSQL + Grafana
├── producer/
│   ├── producer.py             # Producteur Python (NSL-KDD → Kafka)
│   └── KDDTrain+.txt           # Dataset NSL-KDD (inclus dans le repo)
├── spark/
│   └── stream_processor.py     # Job Spark Structured Streaming
├── data/
│   └── init.sql                # Script d'initialisation PostgreSQL
└── grafana/
    ├── provisioning/
    │   ├── datasources/postgres.yml   # Datasource auto-provisionnée
    │   └── dashboards/dashboard.yml   # Provider de dashboards
    └── dashboards/
        └── security_dashboard.json    # Dashboard 5 panels
```

### 8.2 Commandes de Démarrage Rapide

```
# 1. Cloner le dépôt
git clone https://github.com/soulaimane-22/Projet_BigData.git
cd Projet_BigData
```

```
# 2. Télécharger le dataset
cd producer
wget https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+.txt
cd ..

# 3. Lancer toute l'infrastructure (Kafka, PostgreSQL, Grafana, Spark,
Producteur) docker compose up -d docker ps # vérifier que les 5 conteneurs
sont Up
# 4. Créer le topic Kafka
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --create \
  --topic security-events --partitions 1 --replication-factor 1



# 5. Dashboard Grafana
# Ouvrir http://localhost:3000 — login: admin / admin
```

## 8.3 Références


- Apache Kafka Documentation — https://kafka.apache.org/documentation/
- Apache Spark Structured Streaming Guide — https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- NSL-KDD Dataset — Tavallaee, M. et al. (2009). A detailed analysis of the KDD CUP 99 data set.
- Grafana Documentation — https://grafana.com/docs/
- Docker Compose Reference — https://docs.docker.com/compose/
- Jay Kreps — Questioning the Lambda Architecture (2014) — https://www.oreilly.com/radar/questioning-the-lambda-architecture/
- SDKMAN! — https://sdkman.io/